

PETCARE+ API SPECIFICATION

Project Name: PetCarePlus

Version: 1.0

Date: November 18, 2025

Status: Final / Release

Technology Stack: ASP.NET Core 9, React, MySQL

1. OVERVIEW

PetCarePlus is a comprehensive **pet care management system**, designed using a **Modular Monolith architecture**. It facilitates seamless interaction between **Pet Owners, Veterinarians, and System Administrators**, providing unified management of pets, clinical records, appointments, and inventory.

Technical Architecture

- **Backend Framework:** ASP.NET Core 9 (.NET 9)
- **Frontend:** React + TypeScript + Vite
- **Database:** MySQL 8 (Entity Framework Core)
- **Authentication:** ASP.NET Identity + JWT Bearer
- **Architecture Style:** Modular Monolith (feature-based boundaries)

2. AUTHENTICATION & SECURITY

2.1 Authentication Strategy

The PetCarePlus API uses **JSON Web Tokens (JWT)** for stateless, server-independent authentication.

Authorization Header Requirement

Protected endpoints must include:

Authorization: Bearer <your_jwt_token>

Token Handling (Client-Side)

- Tokens are stored in **localStorage**.
 - Storage Key: APP_AT
 - Token includes user roles, user ID, and expiration timestamp.
-

2.2 Role-Based Access Control (RBAC)

Role	Code	Permission Summary
Owner	OWNER	Register pets, request appointments, view own pets & records.
Veterinarian	VET	Manage appointments, create medical records, prescribe medication.
Administrator	ADMIN	Full system control, user management, master overrides.

RBAC is enforced both at:

- Controller level (via [Authorize(Roles="")])
 - Business logic layer for sensitive operations
-

3. SYSTEM ENVIRONMENTS

Development

- **API Base:** <http://localhost:5002>
- **Frontend:** <http://localhost:5173>
- **Swagger Docs:** <http://localhost:5002/swagger>

Production

- **API Base:** <https://api.petcareplus.com>
 - **Frontend:** <https://petcareplus.com>
-

4. API ENDPOINTS

4.1 Authentication Module

4.1.1 Register Owner

- **Method:** POST
- **Endpoint:** /api/auth/register-owner
- **Access:** Public

Payload

```
{  
  "fullName": "John Smith",  
  "email": "john.smith@example.com",  
  "password": "SecurePass123!"
```

```
}
```

Success Response (201)

```
{  
  "message": "Registration successful"  
}
```

4.1.2 Login

- **Method:** POST
- **Endpoint:** /api/auth/login
- **Access:** Public

Payload

```
{  
  "email": "admin@petcare.com",  
  "password": "AdminPass123"  
}
```

Success Response (200)

Returns: accessToken, expiresAt, userId, roles.

4.1.3 Get Current User Profile

- **Method:** GET
- **Endpoint:** /api/auth/me
- **Access:** Protected

Retrieves authenticated user's profile and roles.

4.2 User Management Module

4.2.1 Update Profile

- **Method:** PUT
- **Endpoint:** /api/users/me
- **Payload:**

```
{
```

```
"fullName": "Updated Name",  
"email": "updated@example.com"  
}
```

4.3 Pet Management Module

4.3.1 Get All Pets

- **Method:** GET
 - **Endpoint:** /api/pets
 - **Access:** Admin, Vet
 - **Description:** Retrieves a list of all pets in the system.
-

4.3.2 Get Pet by ID

- **Method:** GET
 - **Endpoint:** /api/pets/{id}
 - **Access:** Protected
 - **Notes:** Owners can only access their own pets.
-

4.3.3 Create Pet (Admin Only)

- **Method:** POST
- **Endpoint:** /api/pets
- **Access:** Admin

Payload

```
{  
  "name": "Max",  
  "species": "Dog",  
  "breed": "Golden Retriever",  
  "dateOfBirth": "2020-03-15",  
  "color": "Golden",  
  "weight": 30.5,  
  "medicalNotes": "Allergic to chicken",  
  "ownerUserId": "owner-guid"
```

```
}
```

4.3.4 Update Pet

- **Method:** PUT
- **Endpoint:** /api/pets/{id}
- **Access:** Admin Only

4.3.5 Assign Pet

- **Method:** POST
- **Endpoint:** /api/pets/{id}/assign
- **Description:** Transfers pet ownership to another user.

4.4 Appointment Module

4.4.1 Create Appointment Request

- **Method:** POST
- **Endpoint:** /api/appointments
- **Access:** Owner, Admin

Payload

```
{  
  "petId": "pet-guid-1",  
  "requestedDateTime": "2025-11-25T10:00:00Z",  
  "reasonForVisit": "Annual Checkup"  
}
```

4.4.2 Update Appointment Status

- **Method:** PUT
- **Endpoint:** /api/appointments/{id}/status
- **Access:** Admin Only

Example Payload (Approve)

```
{  
  "status": "Approved",
```

```
"adminNotes": "Assigned to Dr. Smith",  
"vetUserId": "vet-guid"  
}
```

4.4.3 Appointment Summary Report

- **Method:** GET
- **Endpoint:** /api/appointments/summary-report
- **Access:** Admin

Returns statistics such as:

- upcomingAppointmentsCount
 - busiestDayOfWeek
 - completedAppointments
 - cancellationRates
-

4.5 Notification Module

- **GET /api/notifications** — Retrieve notifications for logged-in user.
 - **GET /api/notifications/unread-count** — Returns unread badge count.
 - **PUT /api/notifications/{id}/read** — Mark a notification as read.
-

4.6 Clinical Modules

4.6.1 Medical Records

- **POST /api/medicalrecords** — Create new medical record (*Vet/Admin*)
 - **GET /api/medicalrecords/pet/{petId}** — Fetch pet medical history
-

4.6.2 Vaccinations

- **POST /api/vaccinations**

Sample Payload

```
{  
  "petId": "pet-guid",  
  "vaccineName": "Rabies",  
  "nextDueDate": "2026-11-18"
```

}

4.6.3 Prescriptions

- **POST /api/prescriptions**

Required Fields:

- medicationName
- dosage
- frequency
- durationDays

4.7 Inventory Module

- **GET /api/inventory** — List all inventory items
- **GET /api/inventory/search?q=term** — Search items
- **POST /api/inventory** — Add new stock (*Admin/Vet*)
- **POST /api/inventory/upload-photo** — Upload inventory photo (*multipart/form-data*)

4.8 Admin User Management

4.8.1 Create Veterinarian

- **Method:** POST
- **Endpoint:** /api/admin/users/vets
- **Access:** Admin Only

Creates a Veterinarian user with role **VET**.

4.8.2 List Users

- **Method:** GET
- **Endpoint:** /api/admin/users
- **Supported Query Params:**
 - role
 - search
 - page
 - pageSize

5. DATA MODELS

User

```
{  
  id: string;  
  fullName: string;  
  email: string;  
  roles: string[]; // ["OWNER", "VET", "ADMIN"]  
  accountStatus: "Active" | "Inactive";  
}
```

Appointment

```
{  
  id: string;  
  petId: string;  
  status: "Pending" | "Approved" | "Cancelled" | "Completed" | "NoShow";  
  requestedDateTime: DateTime;  
  vetUserId?: string;  
}
```

Inventory Item

```
{  
  id: string;  
  name: string;  
  category: string;  
  quantity: number;  
  unit: string;  
  price: number;  
}
```

6. ERROR HANDLING

The API follows **RFC 7807 – Problem Details for HTTP APIs**.

Standard Error Response

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Error Title",
  "status": 400,
  "detail": "Detailed explanation",
  "errors": {
    "field": ["Specific validation error"]
  }
}
```

Common HTTP Status Codes

- **200 OK** — Operation successful
 - **201 Created** — Resource successfully created
 - **400 Bad Request** — Validation failed
 - **401 Unauthorized** — Missing/invalid token
 - **403 Forbidden** — Not enough permissions
 - **404 Not Found** — Requested ID not found
 - **500 Internal Server Error** — Unhandled server error
-

7. TESTING & QUALITY ASSURANCE

7.1 Test Data

- **Admin Login:**
 - Email: admin@petcare.com
 - Password: AdminPass123

7.2 Tools

- **Swagger UI** for interactive API exploration
- **Postman Collection:** /docs/PetCarePlus.postman_collection.json

7.3 Automation

- **Backend:** dotnet test — Integration Tests
- **Frontend:** npm run test:e2e — Playwright end-to-end tests